

SIMATS ENGINEERING
ASSESSMENT TOOL 3 – CASE STUDY

COURSE CODE: ITA14

COURSE NAME: Ethical Hacking

COURSE OUTCOME COVERED

CO4: Implement network security testing methods by performing web server attacks, analyzing web application vulnerabilities, and assessing wireless network security using Kali Linux. (BT4, BT5)

Assessment Tool Weightage: 35%

QUESTIONS: 5

Total Marks:100

-
1. A mid-sized e-commerce company hosts its web application on an Apache server running on a Linux environment. Recently, customers reported unauthorized transactions and abnormal account activities. Upon investigation, the IT team discovered suspicious HTTP requests in server logs, including attempts to access hidden directories and injection of malicious scripts into input fields. Additionally, the server is running an outdated version with known vulnerabilities, and no Web Application Firewall (WAF) is implemented. As a cybersecurity analyst, you are tasked with conducting a comprehensive security assessment using Kali Linux. Analyze the possible attack vectors exploited by the attacker, including web server vulnerabilities, input validation issues, and outdated software components. Explain in detail how you would perform reconnaissance, vulnerability scanning, and exploitation using appropriate Kali Linux tools. Further, propose a layered defense strategy that includes server hardening, secure coding practices, and network-level protections to prevent such incidents in the future.
 2. A financial services web application allows users to log in, view account details, and perform transactions. A recent penetration test revealed that the application is vulnerable to SQL injection and Cross-Site Scripting (XSS) attacks. Error messages expose database structure information, and user inputs are not properly sanitized. Attackers have exploited these weaknesses to extract sensitive customer data and inject malicious scripts into web pages viewed by other users. You are assigned to perform a detailed security audit of the application. Question: Examine how SQL injection and XSS vulnerabilities can be exploited in this scenario. Describe a step-by-step methodology using Kali Linux tools such as Burp Suite and SQLmap to identify and validate these vulnerabilities. Evaluate the impact of such attacks on confidentiality, integrity, and availability (CIA triad). Recommend comprehensive mitigation strategies, including input validation, output encoding, secure session handling, and implementation of security controls.
 3. A corporate office provides wireless internet access to employees using WPA2-PSK authentication. Recently, several employees reported unusual login prompts and slow network performance. A security investigation revealed the presence of a rogue access point (Evil Twin) mimicking the official network SSID. Attackers used this setup to capture login credentials and perform man-in-the-middle (MITM) attacks. You are tasked with analyzing the breach and strengthening wireless security. Explain how the attacker successfully executed the Evil Twin attack and intercepted sensitive data. Describe how Kali Linux tools such as Aircrack-ng and Wireshark can be used to detect and analyze such attacks. Evaluate the weaknesses in the existing wireless security configuration. Propose robust countermeasures, including secure authentication mechanisms, network monitoring, and user awareness strategies.

4. An organization's internal network hosts a web-based employee management system accessible over Wi-Fi. The wireless network uses weak passwords, and the web application lacks proper authentication and session management. An attacker gains access to the Wi-Fi network, performs reconnaissance, and exploits vulnerabilities in the web application to escalate privileges and access confidential employee data. You are assigned to perform a full penetration testing assessment. Analyze the attack chain starting from wireless network compromise to web application exploitation. Describe each phase of the attack, including reconnaissance, scanning, exploitation, and privilege escalation. Demonstrate how Kali Linux tools can be used in each phase. Propose a comprehensive security framework that addresses both wireless and web application vulnerabilities, ensuring defense-in-depth.
5. A university campus experiences frequent Wi-Fi disruptions during peak hours. Investigation reveals repeated deauthentication attacks targeting access points, causing legitimate users to disconnect. The attacker uses automated tools to continuously send deauth packets, leading to denial of service. You are tasked with identifying the cause and implementing solutions. Explain how deauthentication attacks are carried out in wireless networks and how attackers exploit protocol weaknesses. Describe how Kali Linux tools can be used to simulate and analyze such attacks in a controlled environment. Evaluate the impact on network availability and user productivity. Recommend effective mitigation strategies, including network configuration changes, intrusion detection systems, and secure wireless protocols.

Code of Conduct:

I Aluri Poojitha[192511230]_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity difficult to understand

Question 1 — E-Commerce Apache Server Compromise: Security Assessment

Scenario: A mid-sized e-commerce company hosts its web application on an outdated Apache server on Linux, with no Web Application Firewall (WAF). Server logs show attempts to access hidden directories and injected malicious scripts, and customers have reported unauthorized transactions.

1. Possible Attack Vectors Exploited

- Outdated Apache/software components — unpatched CVEs allow known exploits (e.g., RCE, directory traversal) to be run against the server.
- Hidden/unlinked directory exposure — attacker enumerated backup folders, admin panels, or config files (e.g., /backup, /admin, /.git) via forced browsing.
- Improper input validation — form fields and URL parameters accept unsanitised input, enabling script/SQL injection.
- Cross-Site Scripting (XSS) — malicious JavaScript injected into input fields is stored/reflected and executed in other users' browsers, enabling session/cookie theft.
- Missing WAF — no layer to detect/block malicious payloads (SQLi, XSS, path traversal) before they reach the application.

- Weak session management — stolen cookies/tokens reused to hijack authenticated sessions, explaining the 'unauthorized transactions.'

2. Security Assessment Methodology on Kali Linux

2.1 Reconnaissance

Goal: passively and actively gather information about the target — domain details, live hosts, open ports, and server banners — without yet attacking it.

- whois and nslookup / dig — domain registration and DNS records.
- nmap — host discovery, open ports, service/version detection, and OS fingerprinting.
- theHarvester — emails, subdomains, and related OSINT.
- Wappalizer / whatweb — identify web server, CMS, and framework versions.

```
poojitha@kali:~$ whois eshop-target.com | grep -i "registrar\|creation"
poojitha@kali:~$ nmap -sV -O -p- eshop-target.com
Starting Nmap 7.94 ( https://nmap.org )
Nmap scan report for eshop-target.com (203.0.113.10)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.6p1
80/tcp    open  http     Apache httpd 2.4.29 ((Ubuntu))
443/tcp   open  ssl/http Apache httpd 2.4.29
3306/tcp  filtered mysql
OS details: Linux 4.15 - 5.4
poojitha@kali:~$ whatweb eshop-target.com
eshop-target.com [200 OK] Apache[2.4.29], PHP[7.2.24], WordPress[5.4]
```

2.2 Vulnerability Scanning

Goal: match the fingerprinted software versions against known vulnerabilities and probe the web application/server configuration for weaknesses.

- Nikto — web server misconfiguration, outdated software, dangerous files/directories.
- dirb / gobuster — brute-force hidden directories and backup files seen in the logs.
- OWASP ZAP / Burp Suite (passive scan) — crawl the app and flag injection points, missing security headers, and outdated libraries.
- searchsploit — cross-reference the Apache/PHP/CMS version against known public exploits.

```
poojitha@kali:~$ nikto -h https://eshop-target.com
- Server: Apache/2.4.29 (Ubuntu)
- The X-Frame-Options header is not set.
- /backup/: Directory indexing found.
- /admin/config.php.bak: Backup file may expose credentials.
- OSVDB-3268: Apache 2.4.29 - multiple known CVEs for this version.
poojitha@kali:~$ gobuster dir -u https://eshop-target.com -w
/usr/share/wordlists/dirb/common.txt
/admin                (Status: 200)
/backup               (Status: 200)
/uploads              (Status: 200)
/config.php.bak       (Status: 200)
```

```
poojitha@kali:~$ searchsploit apache 2.4.29
Apache 2.4.29 mod_cgi - Remote Code Execution | linux/remote/xxxx.py
```

2.3 Exploitation (Controlled / Authorised Test)

Goal: validate that the discovered weaknesses are genuinely exploitable, under an authorised penetration-testing engagement, so their real-world impact can be reported.

- Burp Suite — intercept and manipulate input-field requests to confirm stored/reflected XSS.
- sqlmap — confirm whether unsanitised parameters allow SQL injection against the order/login forms.
- Metasploit Framework — validate a matched CVE (from searchsploit) against the Apache version in a lab/staging clone, never production.

```
poojitha@kali:~$ sqlmap -u "https://eshop-target.com/product.php?id=1" --batch --level=2
[INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[INFO] GET parameter 'id' appears to be injectable
sqlmap identified the following injection point(s):
  Parameter: id (GET)  Type: boolean-based blind
poojitha@kali:~$ msfconsole -q -x "search apache 2.4.29; exit"
Matching Modules
=====
exploit/unix/http/apache_mod_cgi_bash_env_exec
```

Note: exploitation is performed only within a written scope/authorisation, ideally against a staging replica, and every step is logged for the final pentest report — never against a live production system without sign-off.

3. Layered Defense Strategy

3.1 Server Hardening

- Patch Apache, PHP, and the CMS to the latest stable versions; subscribe to CVE alerts for the stack.
- Disable directory listing (Options -Indexes) and remove/relocate backup files (.bak, .old) out of the web root.
- Run the web service as a low-privilege user; disable unused modules (e.g., mod_cgi if not required).
- Enforce HTTPS/TLS everywhere and disable weak ciphers/protocols.

3.2 Secure Coding Practices

- Use parameterised queries / prepared statements to eliminate SQL injection.
- Sanitise and encode all user input on output (context-aware output encoding) to prevent XSS.
- Implement strict server-side input validation (whitelisting) in addition to client-side checks.
- Return generic error messages to users; log detailed errors server-side only, never expose stack traces or DB structure.
- Use secure, HttpOnly, SameSite cookies and rotate session tokens after login to reduce session-hijacking risk.

3.3 Network-Level Protections

- Deploy a Web Application Firewall (WAF) such as ModSecurity with the OWASP Core Rule Set to block SQLi/XSS payloads.
- Place an IDS/IPS (e.g., Snort/Suricata) to detect the reconnaissance/scanning patterns seen in the logs.
- Restrict database access with firewall rules; the DB port should never be reachable from the internet.
- Enable centralised logging and alerting (e.g., SIEM) so suspicious HTTP requests trigger real-time alerts.

- Schedule regular vulnerability scans and penetration tests to catch regressions before attackers do.

5. Attack Chain Timeline

Mapping the log evidence to a standard cyber-kill-chain view clarifies how the individual weaknesses were chained together into a full compromise.

1. Reconnaissance	Fingerprinted Apache/PHP version and enumerated hidden paths.	Repeated 404s on /admin, /backup, /.git
2. Scanning	Ran automated scanners to find exploitable endpoints and outdated components.	Bursts of rapid sequential requests, unusual user-agents
3. Weaponisation	Crafted malicious payloads (SQLi strings, script tags) for the input fields.	Encoded payloads in query strings and POST bodies
4. Delivery / Exploitation	Submitted payloads through login/search/product forms; exploited outdated Apache module.	Anomalous POST requests, error stack traces returned
5. Installation	Planted a web shell / malicious script for persistence via an exposed upload or backup path.	New unexpected file in /uploads with recent timestamp
6. Actions on Objective	Extracted customer/session data and performed unauthorised transactions.	Abnormal account activity, unauthorised transactions

6. Impact Analysis — CIA Triad

Confidentiality	Customer PII and payment/session data exposed via injection and hidden backup files.	Regulatory exposure (e.g., PCI-DSS), loss of customer trust
Integrity	Injected scripts and unauthorised transactions altered application behaviour and account/order data.	Financial loss, disputed transactions, data reliability questioned
Availability	Web shell / malicious scripts and potential DoS from scanning can degrade or crash the server.	Downtime, lost sales, SLA breaches

7. Additional Validation — Web Shell & Log Correlation

Beyond the initial scan, the analyst also inspects the flagged upload directory and correlates access logs to confirm whether a backdoor was planted and to build an evidentiary timeline.

```
poojitha@kali:~$ curl -s https://eshop-target.com/uploads/ | grep -i '\.php'
cache_temp_9f21.php
poojitha@kali:~$ file downloaded_cache_temp_9f21.php
downloaded_cache_temp_9f21.php: PHP script, ASCII text, obfuscated (base64_decode)
poojitha@kali:~$ grep 'POST /uploads' /var/log/apache2/access.log | tail -5
203.0.113.55 - - [POST /uploads/cache_temp_9f21.php] 200 512
203.0.113.55 - - [POST /uploads/cache_temp_9f21.php] 200 984
poojitha@kali:~$ msfconsole -q -x "use auxiliary/scanner/http/apache_userdir_enum; exit"
```

The presence of an obfuscated PHP file in an upload directory, repeatedly POSTed to from the same external IP, confirms a planted web shell being used for persistence — matching the 'unauthorized transactions' reported by customers.

8. Risk Assessment Summary

Outdated Apache version (RCE)	High	Critical	Immediate patch
SQL Injection in product/login forms	High	Critical	Immediate fix
Stored/Reflected XSS	Medium	High	Fix in current sprint

Exposed backup files / directory listing	High	Medium	Immediate removal
No WAF present	High	High	Deploy within 1 week

9. Incident Response & Recovery Recommendations

- Isolate the affected server and revoke/rotate all session tokens, API keys, and database credentials immediately.
- Remove the identified web shell and any other unauthorised files; restore the application from a known-clean backup.
- Force a password reset for all customer accounts and notify affected customers per breach-disclosure obligations.
- Preserve logs and the web shell sample for forensic analysis and, if required, law-enforcement reporting.
- Re-scan (Nikto/Nessus) and re-test (Burp/sqlmap) after remediation to confirm the vulnerabilities are closed before returning to production.
- Conduct a post-incident review and update the organisation's secure SDLC and patch-management policy to prevent recurrence.

10. Conclusion

The breach resulted from a combination of an outdated, unpatched Apache server, missing input validation, and the absence of a WAF, which together allowed reconnaissance, directory enumeration, and injection attacks to succeed. A layered defense — combining server hardening, secure coding, and network-level controls — closes each stage of the attack chain (reconnaissance, scanning, exploitation) and is essential to prevent recurrence.